

# FULL TECHNICAL STACK DOCUMENTATION

## Event & Parking Reservation System

ASP.NET Core Web API + SQL Server + Vanilla HTML/CSS/JavaScript

| Document Item        | Value                                                         |
|----------------------|---------------------------------------------------------------|
| Project              | Event & Parking Reservation System                            |
| Document Type        | Full Technical Stack / Implementation Documentation           |
| Basis                | Uploaded BRD v1.3 + uploaded source-code ZIP snapshot         |
| Prepared             | 11 August 2026                                                |
| Implementation Focus | Actual technologies and code currently present in the project |

**Important scope:** This document distinguishes the BRD target architecture from what is actually present in the uploaded source code. Exact versions are taken from EventParking.API.csproj; where the source does not specify a product version (for example, the SQL Server release), the document does not guess it.

# Contents

1. Executive Summary
2. Required Technical Stack Summary
3. System Purpose and Functional Scope
4. Solution Architecture
5. .NET Version and Runtime
6. Backend Technology / Framework
7. Frontend Technology / Framework
8. Database Technology and Data Access
9. Programming Languages and Data Formats
10. APIs and Services Used
11. Libraries, Packages and Supporting Technologies
12. Authentication, Authorization and Security
13. Database Schema and Relationships
14. Functional Modules and Technical Mapping
15. REST API Endpoint Inventory
16. Booking, Seat and Parking Workflow
17. Background Processing and Concurrency
18. Swagger / OpenAPI Documentation
19. Configuration and Environments
20. Project Folder Structure and Key Files
21. Development, Run and Database Setup
22. Git / GitHub and Source Control
23. BRD Alignment and Implementation Notes
24. Deliverables Readiness Checklist
25. Conclusion

## 1. Executive Summary

The Event & Parking Reservation System is a full-stack web application that allows customers to browse events, select event seats, optionally reserve parking, create time-limited bookings, simulate payment, view booking/payment history and notifications, and manage their profile. Administrators manage venues, categories, events, seat maps, parking layouts, customer records and dashboard metrics.

The uploaded implementation follows the BRD's core technology direction: ASP.NET Core Web API on the backend, SQL Server for persistence, and a framework-free HTML/CSS/JavaScript frontend communicating through the Fetch API. The project also adds JWT bearer authentication, Entity Framework Core, BCrypt password hashing, Swagger/OpenAPI, EF migrations and a hosted background worker for expiring unpaid bookings.

**Current implementation baseline:** The project targets .NET 9.0. Entity Framework Core and JwtBearer packages are version 8.0.8, Microsoft.AspNetCore.OpenApi is 9.0.17, Swashbuckle.AspNetCore is 8.0.0 and BCrypt.Net-Next is 4.2.0.

## 2. Required Technical Stack Summary

| Required Item                   | Actual Project Technology                                                                                                                                                                                      | Evidence in Project                                |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|
| .NET Version used               | .NET 9.0 target framework (net9.0)                                                                                                                                                                             | EventParking.API.csproj                            |
| Frontend Technology / Framework | HTML5, CSS3 and vanilla JavaScript; no React/Angular/Vue. Fetch API for HTTP communication.                                                                                                                    | Frontend-UI/*.html, css/*.css, js/*.js             |
| Backend Technology / Framework  | ASP.NET Core Web API with Controllers, Services, DI, JWT authentication and hosted BackgroundService.                                                                                                          | Program.cs, Controllers/, Services/, Workers/      |
| Database used                   | Microsoft SQL Server through a SQL Server Express instance. Development and normal connection strings target EventParkingDb_Dev / EventParkingDb.                                                              | appsettings.json and appsettings.Development.json  |
| Programming Languages           | C# (backend), JavaScript (frontend logic), HTML, CSS, SQL generated/executed via EF Core and migrations. JSON is used for API payloads and configuration.                                                      | Source tree                                        |
| APIs / Services used            | Internal REST API, browser Fetch API, JWT bearer authentication, Swagger/OpenAPI. Payment is simulated. SMTP settings exist but email sending is not implemented.                                              | Controllers, js/api.js, Program.cs, AuthService.cs |
| Other Tools / Libraries         | EF Core 8.0.8, EF Core SQL Server 8.0.8, BCrypt.Net-Next 4.2.0, Swashbuckle 8.0.0, Microsoft.AspNetCore.OpenApi 9.0.17, Google Fonts Inter, Font Awesome 6.4.0, Git/GitHub, Visual Studio solution/migrations. | csproj + frontend HTML + .git metadata             |

### 3. System Purpose and Functional Scope

The system has two principal roles: Customer and Administrator. The source implements a role claim in the JWT and uses ASP.NET Core authorization attributes to protect administrative operations.

| Area          | Customer Capabilities                                                    | Administrator Capabilities                                                                                            |
|---------------|--------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Accounts      | Register, verify email token, login, reset password, view/update profile | Search customers; current API also exposes deactivate/reactivate to any authenticated user - see implementation notes |
| Events        | Browse and filter events                                                 | Create, edit and delete events; manage venue/category master data                                                     |
| Seats         | View visual seat map and select seats                                    | Generate, edit and remove seat-map entries                                                                            |
| Parking       | View parking layout; optionally add one slot to booking                  | Generate, edit and remove parking slots and fees                                                                      |
| Bookings      | Create pending booking, view own bookings, cancel, view hold countdown   | Dashboard statistics; admin cancellation is supported by service logic                                                |
| Payments      | Simulate payment, view history and receipt JSON                          | View system revenue through dashboard metrics                                                                         |
| Notifications | View own notifications and mark as read                                  | Notifications are intended to be generated internally, not manually                                                   |

### 4. Solution Architecture

The project uses a separated frontend/backend architecture. The browser frontend is stored in Frontend-UI and communicates with the ASP.NET Core Web API using HTTP requests through the shared Fetch helper in js/api.js. The backend is layered into controllers, services and Entity Framework Core data access. A repository abstraction exists only for the customer/authentication area, so the actual implementation is a hybrid of Controller -> Service -> EF Core and Controller -> Service -> Repository -> EF Core.

## 4.1 Architecture Diagram and Request Flow

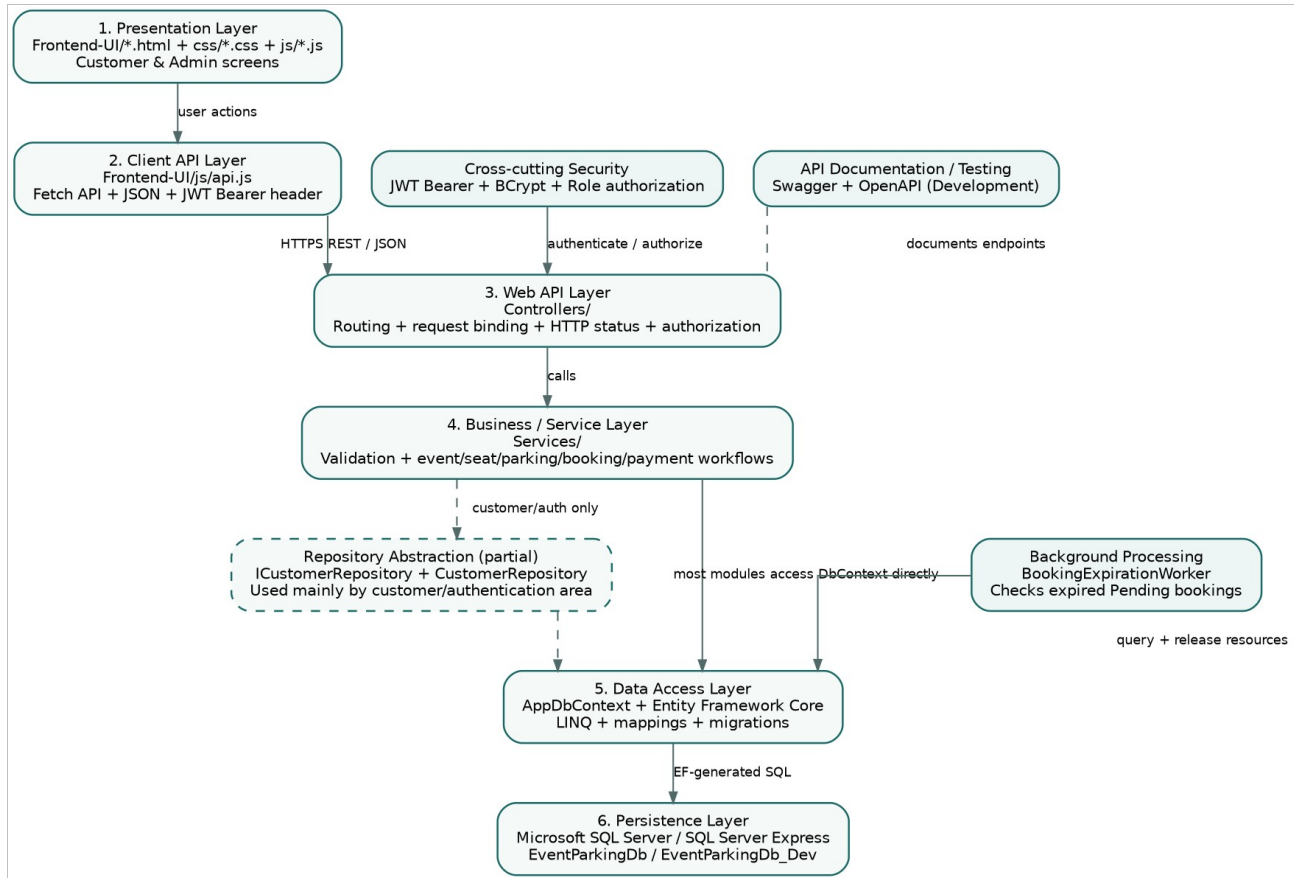


Figure 1. Actual solution architecture and request/data flow found in the uploaded source.

A normal user request moves through the system in this order:

1. The user interacts with an HTML page in Frontend-UI. Page-specific JavaScript reads form values, selected seats, parking choices or dashboard actions.
2. Frontend-UI/js/api.js sends a Fetch request to the backend API. When a JWT is available, it adds the Authorization: Bearer <token> header.
3. An ASP.NET Core controller receives the request, applies routing/authorization and delegates the operation to a service class.
4. The service applies business rules such as venue overlap validation, seat availability, parking availability, booking holds, payment rules and ownership checks.
5. Most services access AppDbContext directly through Entity Framework Core. Auth/customer persistence can pass through ICustomerRepository and CustomerRepository.
6. Entity Framework Core translates LINQ operations into SQL and reads/writes the SQL Server database.
7. The service returns data to the controller, which sends an HTTP/JSON response back to the browser; JavaScript then updates the page.

## 4.2 Layer-by-Layer Structure

| Layer             | Main Components            | Responsibility                                                                      | Project Location |
|-------------------|----------------------------|-------------------------------------------------------------------------------------|------------------|
| Presentation / UI | HTML, CSS, page JavaScript | Screens, forms, responsive layout, seat/parking grids and browser-side interaction. | Frontend-UI/     |

| Layer                | Main Components                          | Responsibility                                                                                                                 | Project Location                          |
|----------------------|------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|
| Client API           | api.js + Fetch API                       | Builds API URLs, serializes JSON, attaches JWT and handles responses/errors.                                                   | Frontend-UI/js/api.js                     |
| Web API              | 11 controller classes                    | REST routes, request binding, status codes, authentication/role attributes.                                                    | Backend-API/EventParking.API/Controllers/ |
| Business / Services  | 11 service classes                       | Main validation and workflows for customers, venues, events, seats, parking, bookings, payments, notifications and dashboards. | Backend-API/EventParking.API/Services/    |
| Repository (partial) | ICustomerRepository + CustomerRepository | Customer data abstraction used by the customer/authentication area only.                                                       | Interfaces/ + Repositories/               |
| Data Access          | AppDbContext + EF Core                   | DbSets, relationships, LINQ queries, decimal precision, indexes and migrations.                                                | Data/AppDbContext.cs + Migrations/        |
| Database             | SQL Server / SQL Server Express          | Persistent relational data for all application modules.                                                                        | EventParkingDb / EventParkingDb_Dev       |
| Background Worker    | BookingExpirationWorker                  | Periodically finds expired Pending bookings and releases reserved resources.                                                   | Workers/ BookingExpirationWorker.cs       |
| Cross-cutting        | JWT, BCrypt, CORS, Swagger               | Security, browser/API connectivity and API documentation/testing.                                                              | Program.cs + AuthService.cs               |

### 4.3 Important Architecture Note

**Actual pattern used:** The BRD recommends Controllers -> Services -> Repositories. The current source does not use repositories for every module. The accurate description is: Frontend -> Controllers -> Services -> EF Core -> SQL Server, with CustomerRepository inserted only in the customer/authentication path. This distinction is important for technical documentation and viva answers.

## 5. .NET Version and Runtime

The backend project targets net9.0, which means the intended runtime is .NET 9. The project SDK is Microsoft.NET.Sdk.Web, the standard SDK for ASP.NET Core web applications. Nullable reference types and implicit global usings are enabled.

Backend-API/EventParking.API/EventParking.API.csproj - TargetFramework = net9.0; Nullable = enable; ImplicitUsings = enable

| Package                                       | Version | Purpose                                          |
|-----------------------------------------------|---------|--------------------------------------------------|
| Microsoft.AspNetCore.OpenApi                  | 9.0.17  | OpenAPI support for the ASP.NET Core application |
| Microsoft.AspNetCore.Authentication.JwtBearer | 8.0.8   | JWT bearer token authentication                  |
| Microsoft.EntityFrameworkCore                 | 8.0.8   | ORM and LINQ-based data access                   |

| Package                                 | Version | Purpose                            |
|-----------------------------------------|---------|------------------------------------|
| Microsoft.EntityFrameworkCore.SqlServer | 8.0.8   | SQL Server EF Core provider        |
| Microsoft.EntityFrameworkCore.Design    | 8.0.8   | Design-time EF services/migrations |
| Microsoft.EntityFrameworkCore.Tools     | 8.0.8   | EF tooling integration             |
| Swashbuckle.AspNetCore                  | 8.0.0   | Swagger UI and OpenAPI generation  |
| BCrypt.Net-Next                         | 4.2.0   | Password hashing and verification  |

**Version note:** The application target is .NET 9.0 while several referenced Microsoft packages are 8.0.8. This is what the uploaded .csproj currently declares; the document reports it as-is rather than assuming a different version.

## 6. Backend Technology / Framework

The backend is an ASP.NET Core Web API using attribute-routed API controllers. Program.cs registers the database context, services, repository, CORS, JWT authentication, controllers, Swagger and the booking-expiration hosted service through the built-in dependency injection container.

Backend-API/EventParking.API/Program.cs - application startup, DI, CORS, JWT, Swagger and middleware pipeline

Backend-API/EventParking.API/Controllers/ - REST endpoints and HTTP responses

Backend-API/EventParking.API/Services/ - business logic and EF Core queries

Backend-API/EventParking.API/DTOs/ - request/response contracts

Backend-API/EventParking.API/Models/ - EF Core entity classes

Backend-API/EventParking.API/Data/AppDbContext.cs - database context and relationship rules

Backend-API/EventParking.API/Migrations/ - database schema changes

Backend-API/EventParking.API/Workers/BookingExpirationWorker.cs - scheduled booking expiry

The middleware order is HTTPS redirection -> CORS -> Authentication -> Authorization -> controller routing. Swagger is enabled only in the Development environment.

### 6.1 Dependency Injection

The following services are registered as scoped: AuthService, CustomerService, EventService, SeatService, VenueService, CategoryService, ParkingService, BookingService, PaymentService, DashboardService and NotificationService. CustomerRepository is registered against ICustomerRepository.

BookingExpirationWorker is registered as a hosted service.

### 6.2 Architectural Pattern

The BRD asks for Controllers -> Services -> Repositories. The current source fully uses this pattern only for customer/authentication persistence. Venue, category, event, seat, parking, booking, payment, dashboard and notification services access AppDbContext directly. Therefore the actual architecture is best described as Controller -> Service -> EF Core, with a repository abstraction in the customer module.

## 7. Frontend Technology / Framework

The frontend is built without a JavaScript framework. Pages are ordinary HTML documents styled with project CSS and driven by vanilla JavaScript. This matches the BRD requirement to avoid React, Angular and Vue and to integrate exclusively through Fetch API calls.

| Frontend Item | Implementation                                                                                                                                                    |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HTML          | Separate pages for login, registration, events, customer dashboard, seat selection, parking selection, payment, notifications, profile and administrator screens. |
| CSS           | Custom CSS in auth.css, admin.css and events.css; pages use responsive layouts and visual state classes.                                                          |
| JavaScript    | Vanilla browser JavaScript; DOM manipulation, URLSearchParams, localStorage, JSON serialization, timers and Fetch.                                                |
| API client    | js/api.js defines BASE_URL and apiFetch(), adds Content-Type: application/json and JWT Authorization when available.                                              |
| UI assets     | Google Fonts (Inter) and Font Awesome 6.4.0 are loaded by several pages through CDN links.                                                                        |
| State         | JWT stored as localStorage key jwtToken. Booking/event IDs are passed between pages via query-string parameters.                                                  |

### 7.1 Main Frontend Pages

| Page                                                           | Purpose                                           |
|----------------------------------------------------------------|---------------------------------------------------|
| login.html / register.html                                     | Authentication entry and new account registration |
| verify-email.html / forgot-password.html / reset-password.html | Account security flows                            |
| events.html                                                    | Customer-facing event listing and filtering       |
| seat-map.html                                                  | Clickable seat map and booking creation           |
| parking-map.html                                               | Optional parking selection                        |
| payment.html                                                   | Hold countdown and simulated payment              |
| customer-dashboard.html                                        | Booking/payment history and cancellation actions  |
| notifications.html                                             | Notification list and mark-as-read UI             |
| profile.html                                                   | Customer profile view/update                      |
| admin-dashboard.html                                           | System-wide metrics                               |
| admin-events.html                                              | Event CRUD                                        |
| admin-venues.html                                              | Venue/category management                         |
| admin-seats.html                                               | Seat-map administration                           |
| admin-parking.html                                             | Parking-layout administration                     |
| admin-customers.html                                           | Customer search/status management                 |



## 8. Database Technology and Data Access

The application uses Microsoft SQL Server through the EF Core SQL Server provider. The connection string points to a SQL Server Express named instance and uses Windows trusted authentication, Multiple Active Result Sets and TrustServerCertificate. Separate database names are configured for normal and Development environments.

| Environment | Database Configuration in Source                  |
|-------------|---------------------------------------------------|
| Default     | SQL Server Express instance -> EventParkingDb     |
| Development | SQL Server Express instance -> EventParkingDb_Dev |

The exact SQL Server product release (for example 2019 or 2022) is not declared in the project files, so this documentation identifies the database as SQL Server Express without inventing a release version.

### 8.1 Entity Framework Core

- Code-first entity classes are defined in Models/.
- ApplicationDbContext exposes DbSet<T> properties for Customers, Venues, EventCategories, Events, Seats, Bookings, BookingSeats, ParkingSlots, ParkingReservations, Payments and Notifications.
- Migrations record the schema evolution from the initial customer table through events, seats, bookings, parking, payments and notifications.
- LINQ queries are translated by EF Core into SQL Server queries.
- Transactions are explicitly used in BookingService.CreateBookingAsync() so seat and optional parking updates commit together.

### 8.2 Decimal and Uniqueness Rules

AppDbContext configures decimal(18,2)-style precision for seat prices, booking totals, parking fees, parking reservation snapshots and payment amounts. BookingSeat uses a composite primary key (BookingId, SeatId). Payment.BookingId and ParkingReservation.BookingId are unique indexes, enforcing at most one payment and at most one parking reservation per booking.

## 9. Programming Languages and Data Formats

| Language / Format | Where Used           | Why It Is Used                                                                                                                 |
|-------------------|----------------------|--------------------------------------------------------------------------------------------------------------------------------|
| C#                | ASP.NET Core backend | Controllers, services, DTOs, models, EF Core mappings, background worker and authentication logic.                             |
| JavaScript        | Browser frontend     | Fetch requests, DOM updates, seat/parking selection, login token handling, booking/payment workflows.                          |
| HTML5             | Frontend pages       | Page structure, forms, navigation, tables and interactive containers.                                                          |
| CSS3              | Frontend styling     | Responsive UI, grids, Flexbox, status states, admin/customer layouts.                                                          |
| SQL               | Database layer       | SQL Server persistence; SQL is primarily generated through EF Core and migrations rather than handwritten application queries. |

| Language / Format | Where Used          | Why It Is Used                                                                           |
|-------------------|---------------------|------------------------------------------------------------------------------------------|
| JSON              | API + configuration | Request/response bodies, appsettings.json, launchSettings.json and client serialization. |

## 10. APIs and Services Used

### 10.1 Internal REST API

The main service interface is the project's own REST API. Controllers expose resource-style routes for authentication, customers, venues, categories, events, seats, parking, bookings, payments, notifications and dashboard metrics. JSON is the data interchange format.

### 10.2 Browser Fetch API

Frontend code does not call the database directly. `js/api.js` wraps `window.fetch()`, prefixes requests with `https://localhost:7283/api`, adds JSON headers, injects the JWT Bearer token from `localStorage`, reads response text and then parses JSON when present.

### 10.3 JWT Authentication Service

`AuthService` creates signed JWTs containing the customer ID as the subject, email, and role claim. The token lifetime is two hours. ASP.NET Core `JwtBearer` validation checks issuer, audience, lifetime and signing key.

### 10.4 Payment Service

No real payment provider such as Stripe or PayPal is integrated. Payment processing is a simulation stored in the Payments table. A successful payment creates a receipt number and changes the booking status to Confirmed.

### 10.5 Email / SMTP Configuration

`appsettings.json` contains SMTP configuration placeholders and `AuthService` contains token-generation logic for verification and password reset. However, there is no SMTP client or email-sending service implementation in the uploaded source; `ForgotPasswordAsync()` contains a comment indicating where email integration should be added.

### 10.6 Notifications

Notifications are stored in SQL Server and exposed by REST endpoints. They are not real-time push notifications. `PaymentService` creates a notification internally after successful payment. SignalR/push delivery is not used.

## 11. Libraries, Packages and Supporting Technologies

| Technology            | How It Is Used in This Project                                                                                |
|-----------------------|---------------------------------------------------------------------------------------------------------------|
| Entity Framework Core | Object-relational mapping, LINQ queries, migrations, DbContext, relational constraints and SQL Server access. |
| BCrypt.Net-Next       | Hashes registration passwords and verifies login passwords; plain-text passwords are not persisted.           |

| Technology                                 | How It Is Used in This Project                                                                      |
|--------------------------------------------|-----------------------------------------------------------------------------------------------------|
| JwtBearer / Microsoft.IdentityModel.Tokens | Validates bearer tokens and signs tokens with HMAC SHA-256.                                         |
| Swashbuckle.AspNetCore                     | Generates Swagger/OpenAPI documentation and Swagger UI.                                             |
| ASP.NET Core built-in DI                   | Injects DbContext, repository, services and hosted worker dependencies.                             |
| BackgroundService                          | BookingExpirationWorker runs repeatedly to expire unpaid Pending bookings.                          |
| Google Fonts Inter                         | Typography on several frontend pages.                                                               |
| Font Awesome 6.4.0                         | Icons in several admin/customer pages.                                                              |
| Git / GitHub                               | Source control; the ZIP contains a Git repository with main, develop and multiple feature branches. |
| Visual Studio solution                     | EventParking.sln and Visual Studio project metadata show the intended IDE/project workflow.         |

## 12. Authentication, Authorization and Security

| Security Area      | Current Implementation                                                                                                     |
|--------------------|----------------------------------------------------------------------------------------------------------------------------|
| Password hashing   | BCrypt.Net-Next HashPassword() on registration and Verify() on login.                                                      |
| Email verification | Secure random URL-safe token generated with RandomNumberGenerator.GetBytes(32); login blocked until EmailVerified is true. |
| Password reset     | Random token with 60-minute expiration; successful reset replaces password hash and clears reset token.                    |
| JWT                | HMAC-SHA256 signed token; issuer EventParkingAPI, audience EventParkingUsers, two-hour lifetime.                           |
| Roles              | Role stored on Customer and added to ClaimTypes.Role; [Authorize(Roles="Admin")] protects many admin endpoints.            |
| HTTPS              | UseHttpsRedirection() is enabled.                                                                                          |
| CORS               | AllowFrontend policy currently permits any origin, method and header.                                                      |
| Swagger auth       | Swagger defines a Bearer security scheme so JWT-protected routes can be tested from Swagger UI.                            |

**Production security items to change:** Do not keep the JWT signing key or environment-specific database values in committed appsettings.json. Move secrets to user-secrets/environment variables or a secrets manager. Restrict CORS to trusted frontend origins. The BRD calls for hashed verification/reset tokens, but the current source stores them as raw token strings.

## 13. Database Schema and Relationships

The database is modeled with Entity Framework Core entity classes and `AppDbContext`. The current source contains 11 entity/table models. The diagram below separates enforced database foreign keys from logical application references. Solid relationships represent actual EF/SQL foreign-key relationships; dashed relationships represent values used by application logic without a database FK constraint.

### 13.1 Actual ER / Relationship Diagram

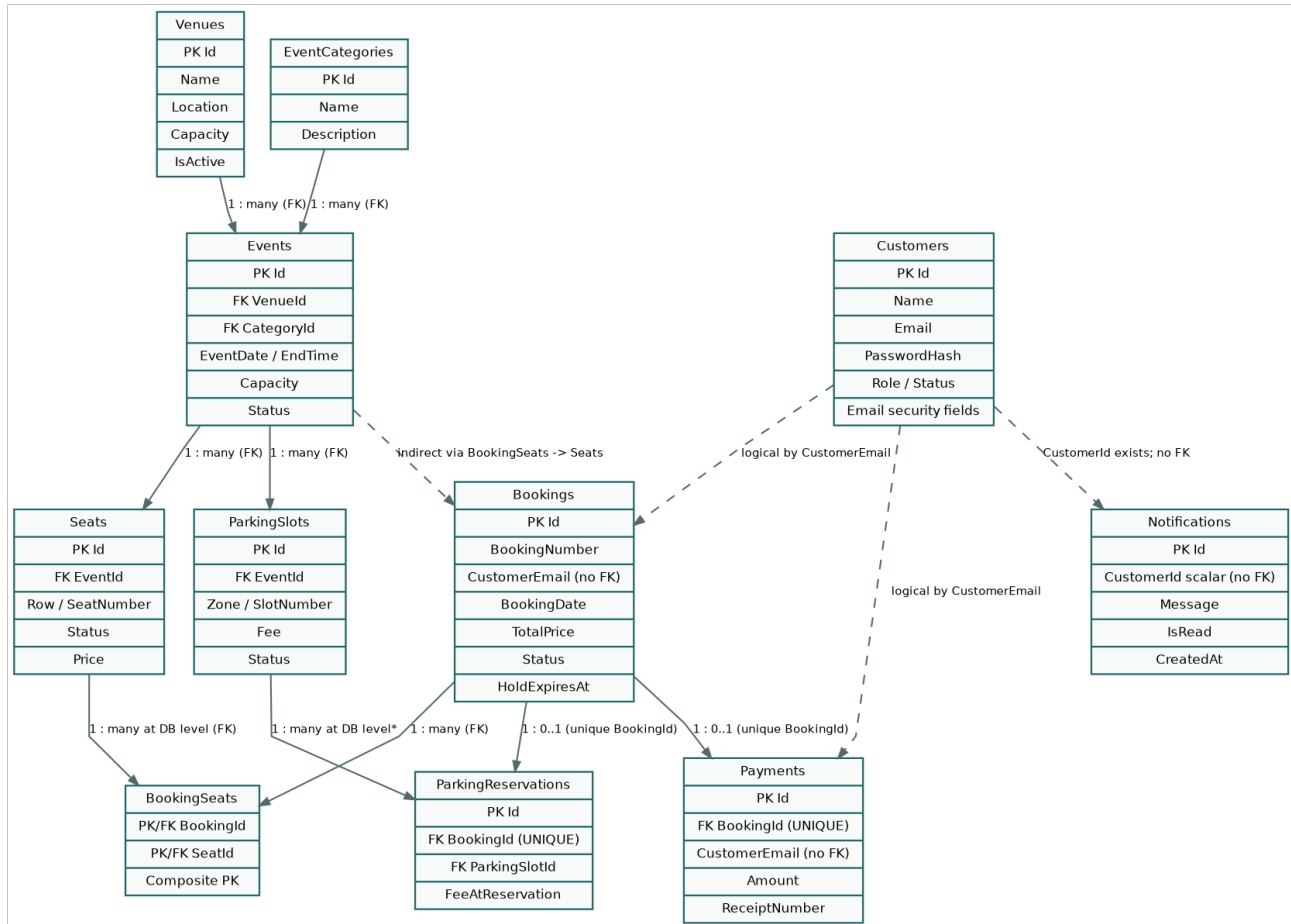


Figure 2. Actual schema relationships from the current models, `AppDbContext` and migration snapshot. \*`ParkingSlotId` is not unique in `ParkingReservations`, so slot exclusivity is enforced by application status checks rather than a database one-to-one constraint.

### 13.2 Table Structure Breakdown

| Table           | Primary / Foreign Keys                                           | Main Fields                                                                                       | Purpose                                                                  |
|-----------------|------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| Customers       | PK: Id                                                           | Name, Email, Phone, PasswordHash, Status, Role, EmailVerified, token fields, CreatedAt, UpdatedAt | Stores customer accounts, authentication/security state and role/status. |
| Venues          | PK: Id                                                           | Name, Location, Capacity, IsActive                                                                | Master data for physical event venues.                                   |
| EventCategories | PK: Id                                                           | Name, Description                                                                                 | Master list used to classify events.                                     |
| Events          | PK: Id; FK: VenueId -> Venues; FK: CategoryId -> EventCategories | Title, Description, EventDate, EndTime, Capacity, Status, ImageUrl, CreatedAt                     | Event catalogue; links every event to one venue and one category.        |

| Table               | Primary / Foreign Keys                                                                 | Main Fields                                                                  | Purpose                                                                                                              |
|---------------------|----------------------------------------------------------------------------------------|------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| Seats               | PK: Id; FK: EventId -> Events                                                          | Row, SeatNumber, Status, Price                                               | Physical/selectable seats for a specific event.                                                                      |
| Bookings            | PK: Id; no Customer/Event FK                                                           | BookingNumber, CustomerEmail, BookingDate, TotalPrice, Status, HoldExpiresAt | Booking header and temporary hold state. Customer is stored by email; event is derived indirectly from booked seats. |
| BookingSeats        | Composite PK: BookingId + SeatId; FKs to Bookings and Seats                            | BookingId, SeatId                                                            | Join table connecting one booking to one or more seats.                                                              |
| ParkingSlots        | PK: Id; FK: EventId -> Events                                                          | Zone, SlotNumber, Fee, Status                                                | Parking spaces configured for an event.                                                                              |
| ParkingReservations | PK: Id; FK: BookingId -> Bookings; FK: ParkingSlotId -> ParkingSlots; UNIQUE BookingId | FeeAtReservation                                                             | Attaches at most one parking reservation to a booking and snapshots the fee.                                         |
| Payments            | PK: Id; FK: BookingId -> Bookings; UNIQUE BookingId                                    | CustomerEmail, Amount, PaymentDate, Status, ReceiptNumber                    | Stores one simulated payment per booking.                                                                            |
| Notifications       | PK: Id; CustomerId has no FK                                                           | Message, IsRead, CreatedAt                                                   | Stores customer notifications; CustomerId is only a scalar value in the current schema.                              |

### 13.3 Relationship Breakdown

| Relationship                      | Cardinality in Current Schema    | How It Is Enforced / Used                                                                                                                                                                         |
|-----------------------------------|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Venue -> Event                    | 1 : many                         | Events.VenueId is a real FK. Delete behavior is Restrict.                                                                                                                                         |
| EventCategory -> Event            | 1 : many                         | Events.CategoryId is a real FK. Delete behavior is Restrict.                                                                                                                                      |
| Event -> Seat                     | 1 : many                         | Seats.EventId is a real FK; configured with cascade delete.                                                                                                                                       |
| Event -> ParkingSlot              | 1 : many                         | ParkingSlots.EventId is an EF relationship/FK.                                                                                                                                                    |
| Booking <-> Seat                  | many : many through BookingSeats | BookingSeats has a composite PK (BookingId, SeatId) and FKs to both tables. Seat reuse across different bookings is prevented mainly by seat status/business logic, not by a unique SeatId index. |
| Booking -> ParkingReservation     | 1 : 0..1                         | ParkingReservations.BookingId has a unique index, so a booking can have at most one parking reservation.                                                                                          |
| ParkingSlot -> ParkingReservation | 1 : many at database level       | ParkingSlotId is an FK but is not unique. The service checks slot Status to prevent multiple active reservations.                                                                                 |
| Booking -> Payment                | 1 : 0..1                         | Payments.BookingId has a unique index; duplicate payment rows for one booking are blocked.                                                                                                        |
| Customer -> Booking               | logical only                     | Bookings stores CustomerEmail rather than CustomerId; there is no FK to Customers.                                                                                                                |
| Customer -> Payment               | logical only                     | Payments stores CustomerEmail rather than CustomerId; there is no FK to Customers.                                                                                                                |
| Customer -> Notification          | logical/scalar only              | Notifications.CustomerId exists, but the current migration snapshot has no FK constraint to Customers.                                                                                            |

| Relationship     | Cardinality in Current Schema | How It Is Enforced / Used                                                                       |
|------------------|-------------------------------|-------------------------------------------------------------------------------------------------|
| Event -> Booking | indirect only                 | Bookings has no EventId. Event details are derived by Booking -> BookingSeats -> Seat -> Event. |

### 13.4 Schema Design Notes Against the BRD

The BRD proposes direct CustomerId and EventId relationships on Bookings. The current implementation differs: it stores CustomerEmail and derives the event through selected seats. This works for the present code, but it weakens referential integrity and makes some queries more complex. The current Notifications table also lacks a Customer FK, and ParkingReservations does not have a unique constraint on ParkingSlotId. These are implementation gaps, not assumptions in this document.

## 14. Functional Modules and Technical Mapping

| Module                            | Backend                                          | Frontend                                      | Core Technical Logic                                                  |
|-----------------------------------|--------------------------------------------------|-----------------------------------------------|-----------------------------------------------------------------------|
| Authentication / Account Security | AuthController, AuthService, CustomerRepository  | login/register/verify/forgot/reset pages      | BCrypt, JWT, role claim, verification/reset tokens                    |
| Customer Management               | CustomersController, CustomerService             | profile.html, admin-customers.html            | Profile DTOs, search, status changes                                  |
| Venue & Category                  | VenuesController, CategoriesController, services | admin-venues.html                             | CRUD and venue-availability checks                                    |
| Event Management                  | EventsController, EventService                   | events.html, admin-events.html                | Filtering, venue overlap validation, event capacity validation        |
| Seat Reservation                  | SeatsController, SeatService                     | seat-map.html, admin-seats.html               | Visual seat grid, seat generation, selection, booked-state protection |
| Parking Reservation               | ParkingController, ParkingService                | parking-map.html, admin-parking.html          | Optional one-slot selection, fixed fee snapshot                       |
| Booking Management                | BookingsController, BookingService               | seat-map -> parking-map -> payment; dashboard | Transaction, booking number, Pending hold, cancellation/release       |
| Payment Simulation                | PaymentsController, PaymentService               | payment.html, customer-dashboard.html         | Payment record, Confirmed status, receipt JSON                        |
| Notifications                     | NotificationsController, NotificationService     | notifications.html (path issue noted later)   | Customer ownership checks, read status, internal creation             |
| Dashboards                        | DashboardController, DashboardService            | admin-dashboard.html, customer-dashboard.html | Aggregate counts, revenue, bookings/payments display                  |

## 15. REST API Endpoint Inventory

| Method | Route                         | Security | Purpose                              |
|--------|-------------------------------|----------|--------------------------------------|
| POST   | /api/auth/register            | Public   | Register customer                    |
| POST   | /api/auth/login               | Public   | Authenticate and issue JWT           |
| GET    | /api/auth/verify-email?token= | Public   | Verify email token                   |
| POST   | /api/auth/forgot-password     | Public   | Create reset token; generic response |

| Method | Route                                        | Security               | Purpose                                                             |
|--------|----------------------------------------------|------------------------|---------------------------------------------------------------------|
| POST   | /api/auth/reset-password                     | Public                 | Reset password                                                      |
| GET    | /api/customers                               | Authenticated          | Search/list customers                                               |
| GET    | /api/customers/{id}                          | Authenticated          | Get customer profile                                                |
| PUT    | /api/customers/{id}                          | Authenticated          | Update profile                                                      |
| DELETE | /api/customers/{id}                          | Authenticated          | Deactivate customer                                                 |
| POST   | /api/customers/{id}/reactivate               | Authenticated          | Reactivate customer                                                 |
| GET    | /api/venues                                  | Public                 | List venues                                                         |
| GET    | /api/venues/{id}                             | Public                 | Get venue                                                           |
| GET    | /api/venues/available                        | Public                 | Availability query                                                  |
| POST   | /api/venues                                  | Admin                  | Create venue                                                        |
| PUT    | /api/venues/{id}                             | Admin                  | Update venue                                                        |
| DELETE | /api/venues/{id}                             | Admin                  | Delete venue                                                        |
| GET    | /api/categories                              | Public                 | List categories                                                     |
| POST   | /api/categories                              | Admin                  | Create category                                                     |
| PUT    | /api/categories/{id}                         | Admin                  | Update category                                                     |
| DELETE | /api/categories/{id}                         | Admin                  | Delete category                                                     |
| GET    | /api/events                                  | Public                 | List/filter events                                                  |
| GET    | /api/events/{id}                             | Public                 | Get event                                                           |
| POST   | /api/events                                  | Admin                  | Create event                                                        |
| PUT    | /api/events/{id}                             | Admin                  | Update event                                                        |
| DELETE | /api/events/{id}                             | Admin                  | Delete event                                                        |
| GET    | /api/events/{eventId}/seats                  | Public                 | Get seat map                                                        |
| POST   | /api/events/{eventId}/seats                  | Admin                  | Generate seat map                                                   |
| PUT    | /api/events/{eventId}/seats/{seatId}         | Admin                  | Edit seat                                                           |
| DELETE | /api/events/{eventId}/seats/{seatId}         | Admin                  | Delete seat                                                         |
| GET    | /api/events/{eventId}/parking-slots          | Public                 | Get parking layout                                                  |
| POST   | /api/events/{eventId}/parking-slots          | Admin                  | Generate parking layout                                             |
| PUT    | /api/events/{eventId}/parking-slots/{slotId} | Admin                  | Edit parking slot                                                   |
| DELETE | /api/events/{eventId}/parking-slots/{slotId} | Admin                  | Delete parking slot                                                 |
| POST   | /api/bookings/{bookingId}/parking            | Authenticated          | Add parking to booking                                              |
| DELETE | /api/bookings/{bookingId}/parking            | Authenticated          | Remove booking parking                                              |
| POST   | /api/bookings                                | Authenticated          | Create pending booking from selected seat IDs + optional parking ID |
| GET    | /api/bookings/{id}/hold-status               | Public in current code | Get remaining hold seconds                                          |
| DELETE | /api/bookings/{id}                           | Authenticated          | Cancel booking                                                      |
| GET    | /api/bookings/my-bookings                    | Authenticated          | Get own bookings                                                    |
| GET    | /api/bookings/{id}/payment                   | Authenticated          | Get amount/payment state                                            |
| POST   | /api/bookings/{id}/payment                   | Authenticated          | Simulate payment                                                    |

| Method | Route                                    | Security            | Purpose                 |
|--------|------------------------------------------|---------------------|-------------------------|
| GET    | /api/payments/customer                   | Authenticated       | Get own payment history |
| GET    | /api/payments/{id}/receipt               | Authenticated       | Get receipt JSON        |
| GET    | /api/notifications/customer/{customerId} | Authenticated owner | Get notifications       |
| PUT    | /api/notifications/{id}/read             | Authenticated owner | Mark read               |
| GET    | /api/dashboard/metrics                   | Admin               | System metrics          |

The above table is based on controller attributes in the uploaded source, not only the BRD. Some BRD endpoints are consolidated, renamed or not yet implemented; see Section 23.

## 16. Booking, Seat and Parking Workflow

8. Customer opens events.html and selects an event.
9. seat-map.html loads GET /api/events/{eventId}/seats. Available seats are selectable; booked seats are disabled.
10. On checkout, the frontend sends selected seat IDs to POST /api/bookings. BookingService opens a database transaction, verifies all requested seats are Available, optionally validates a parking slot, calculates the total and creates a Pending booking.
11. The service marks selected seats Booked, adds BookingSeat rows, optionally adds ParkingReservation, sets HoldExpiresAt and commits the transaction.
12. If parking was not selected during initial booking creation, parking-map.html can call POST /api/bookings/{bookingId}/parking. Parking is optional and the user may skip it.
13. payment.html polls GET /api/bookings/{id}/hold-status to display remaining time and then calls POST /api/bookings/{id}/payment.
14. PaymentService prevents duplicate payment, creates a Payment and receipt number, changes booking status to Confirmed and creates a notification.
15. If payment does not occur before the hold expires, BookingExpirationWorker changes the booking to Expired and releases seats and parking back to Available.

### 16.1 Booking Identifier and Price Calculation

Booking numbers are generated in a human-readable form similar to BKG-<year>-<6-character GUID fragment>. The booking total is the sum of selected Seat.Price values plus ParkingSlot.Fee when parking is reserved. FeeAtReservation snapshots the parking fee so later slot price changes do not retroactively change an existing parking reservation.

## 17. Background Processing and Concurrency

BookingExpirationWorker derives from BackgroundService and runs continuously while the application is active. It waits one minute between checks, finds Pending bookings whose HoldExpiresAt is in the past, changes them to Expired and releases linked seats and parking slots.

Booking creation uses an explicit EF Core transaction, which is a good foundation for keeping seat and parking updates atomic. The BRD also expects concurrency protection against two users attempting the same resource nearly simultaneously.



**Concurrency gap:** The current worker releases expired resources without wrapping the expiry-read/release/save operation in an explicit database transaction. The BRD specifically asks for the expiry job to use the same transaction pattern to avoid overlap with payment. Also, the schema does not currently define a unique index on `ParkingReservation.ParkingSlotId`, so one-slot exclusivity relies on application status checks.

## 18. Swagger / OpenAPI Documentation

`Program.cs` registers `AddEndpointsApiExplorer()` and `AddSwaggerGen()`. Swagger document v1 is titled `EventParking.API`. A Bearer security definition is added so a JWT can be entered in the Swagger UI Authorization control and applied to protected endpoints.

Swagger and Swagger UI are enabled only when `ASPNETCORE_ENVIRONMENT` is `Development`. `launchSettings.json` is configured to open the swagger route automatically. The HTTPS profile uses `https://localhost:7283` and the HTTP profile uses `http://localhost:5157`.

## 19. Configuration and Environments

| Configuration Key                                             | Purpose                      | Current Behaviour                                                                                     |
|---------------------------------------------------------------|------------------------------|-------------------------------------------------------------------------------------------------------|
| <code>ConnectionStrings:DefaultConnection</code>              | SQL Server connection        | Default DB <code>EventParkingDb</code> ; Development override <code>EventParkingDb_Dev</code>         |
| <code>BookingSettings:HoldDurationMinutes</code>              | Pending booking hold         | 15 minutes                                                                                            |
| <code>Jwt:Key / Issuer / Audience</code>                      | Token signing and validation | Symmetric signing key + <code>EventParkingAPI</code> issuer + <code>EventParkingUsers</code> audience |
| <code>EmailSettings</code>                                    | SMTP details                 | Placeholders only; no sending implementation                                                          |
| <code>BusinessRules:BookingHoldMinutes</code>                 | Business-rule config         | Defined as 15, but <code>BookingService</code> reads <code>BookingSettings</code> instead             |
| <code>BusinessRules:EmailVerificationTokenExpiryHours</code>  | Intended token TTL           | Defined 24; <code>AuthService</code> currently hard-codes 24 hours                                    |
| <code>BusinessRules&gt;PasswordResetTokenExpiryMinutes</code> | Intended reset TTL           | Defined 60; <code>AuthService</code> currently hard-codes 60 minutes                                  |

Development logging also enables `Microsoft.EntityFrameworkCore.Database.Command` at Information level, which is useful for seeing SQL commands while debugging.

## 20. Project Folder Structure and Key Files

The structure below is taken from the uploaded ZIP. Build-output and IDE metadata folders such as `bin/`, `obj/` and `.vs/` are intentionally omitted because they are not part of the application architecture. The embedded `.git/` directory is source-control metadata, not application code.

### 20.1 Clean Project Structure

```
Event-Parking-System/
|-- Backend-API/
|   |-- EventParking.sln
|   `-- EventParking.API/
```

```

|-- Controllers/          # 11 REST controllers
| | |-- AuthController.cs
| | |-- BookingsController.cs
| | |-- EventsController.cs
| | |-- SeatsController.cs
| | |-- ParkingController.cs
| | `-- ... customers, venues, categories, payments, notifications, dashboard
|-- DTOs/                # 11 DTO files for request/response contracts
|-- Data/
| | `-- ApplicationDbContext.cs    # EF Core tables, FKs, indexes, precision
|-- Models/              # 11 entity classes / database tables
| | |-- Customer.cs
| | |-- Event.cs / Venue.cs / EventCategory.cs
| | |-- Seat.cs / Booking.cs / BookingSeat.cs
| | `-- ParkingSlot.cs / ParkingReservation.cs / Payment.cs / Notification.cs
|-- Services/            # 11 business/service classes
| | |-- AuthService.cs
| | |-- EventService.cs / SeatService.cs / ParkingService.cs
| | |-- BookingService.cs / PaymentService.cs
| | `-- ... customer, venue, category, dashboard, notification services
|-- Interfaces/
| | `-- ICustomerRepository.cs
|-- Repositories/
| | `-- CustomerRepository.cs
|-- Workers/
| | `-- BookingExpirationWorker.cs
|-- Migrations/          # 10 migrations + designer files + snapshot
|-- Properties/launchSettings.json
|-- Program.cs
|-- appsettings.json
|-- appsettings.Development.json
|-- EventParking.API.csproj
|
|-- Frontend-UI/
| |-- login.html / register.html / forgot-password.html / reset-password.html
| |-- verify-email.html / profile.html / events.html
| |-- seat-map.html / parking-map.html / payment.html / notifications.html
| |-- customer-dashboard.html
| |-- admin-dashboard.html / admin-customers.html / admin-events.html
| |-- admin-seats.html / admin-parking.html / admin-venues.html
| |-- css/
| | |-- auth.css
| | |-- events.css
| | `-- admin.css
| `-- js/
| |-- api.js             # shared Fetch + JWT helper
| |-- events.js / seat-map.js / parking-map.js / payment.js
| |-- customer-dashboard.js / admin-dashboard.js
| `-- auth/              # login/register/profile/admin page scripts
|
|-- .github/workflows/   # GitHub workflow directory
|-- .gitignore
|-- README.md
`-- .git/                # repository metadata in uploaded snapshot

```

## 20.2 Backend Structure Breakdown

| Folder / File               | What It Contains                                                                                                                         | Why It Matters                                                                  |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| Controllers/                | 11 API controllers: auth, customers, venues, categories, events, seats, parking, bookings, payments, notifications and dashboard.        | Defines routes/endpoints and HTTP/authorization boundary.                       |
| DTOs/                       | Request and response DTO classes for authentication, customers, events, bookings, seats, parking, payment, notifications and dashboards. | Keeps API contracts separate from database entities.                            |
| Data/AppDbContext.cs        | DbSet declarations plus EF relationship/index/precision configuration.                                                                   | Central database mapping file; best file for explaining schema relationships.   |
| Models/                     | 11 entity classes that map to the database tables.                                                                                       | Defines persistent data structure.                                              |
| Services/                   | 11 service classes containing core business rules and workflows.                                                                         | Main application logic lives here rather than in controllers.                   |
| Interfaces/ + Repositories/ | ICustomerRepository and CustomerRepository only.                                                                                         | Shows repository abstraction is currently limited to customer/auth persistence. |
| Workers/                    | BookingExpirationWorker.cs.                                                                                                              | Runs scheduled booking-expiry processing.                                       |
| Migrations/                 | 10 migration pairs plus AppDbContextModelSnapshot.cs.                                                                                    | Records database schema evolution and allows EF database creation/update.       |
| Program.cs                  | Service registration, DbContext, CORS, JWT, Swagger and middleware.                                                                      | Application startup/composition root.                                           |
| EventParking.API.csproj     | TargetFramework net9.0 and NuGet package references.                                                                                     | Authoritative file for .NET/package versions.                                   |
| appsettings*.json           | Connection strings, JWT settings, logging and business/email configuration.                                                              | Runtime/environment configuration.                                              |

## 20.3 Frontend Structure Breakdown

| Area                     | Files                                                                                                                                   | Responsibility                                                                                                   |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| Authentication & Account | login.html, register.html, forgot-password.html, reset-password.html, verify-email.html, profile.html + matching scripts under js/auth/ | Registration, login, token flows and profile UI.                                                                 |
| Customer Browsing        | events.html + js/events.js                                                                                                              | Loads/searches events and starts the booking journey.                                                            |
| Seat Selection           | seat-map.html + js/seat-map.js                                                                                                          | Displays selectable event seats and creates/continues a booking.                                                 |
| Parking Selection        | parking-map.html + js/parking-map.js                                                                                                    | Displays optional parking slots and attaches one to the booking.                                                 |
| Payment                  | payment.html + js/payment.js                                                                                                            | Displays total/hold countdown and calls simulated payment endpoints.                                             |
| Customer Dashboard       | customer-dashboard.html + js/customer-dashboard.js                                                                                      | Shows bookings/payment-related customer data.                                                                    |
| Notifications            | notifications.html + js/auth/notifications.js                                                                                           | Displays notifications and read-state actions; note the current HTML/script path inconsistency documented later. |
| Administration           | admin-dashboard/customers/events/seats/parking/venues HTML + admin scripts                                                              | Admin CRUD, layouts and dashboard operations.                                                                    |

| Area       | Files                                       | Responsibility                                                       |
|------------|---------------------------------------------|----------------------------------------------------------------------|
| Shared API | js/api.js                                   | BASE_URL, Fetch wrapper, JSON handling and JWT Authorization header. |
| Styling    | css/auth.css, css/events.css, css/admin.css | Page styling and responsive visual layouts.                          |

## 20.4 Most Important Files for Viva / Technical Explanation

| File                               | What to Explain                                                                                           |
|------------------------------------|-----------------------------------------------------------------------------------------------------------|
| EventParking.API.csproj            | net9.0 target framework and exact NuGet package versions.                                                 |
| Program.cs                         | Dependency injection, SQL Server DbContext, CORS, JWT authentication, Swagger and middleware order.       |
| Data/AppDbContext.cs               | DbSets, real foreign keys, composite key, unique indexes and decimal precision.                           |
| Models/*.cs                        | Actual database entities and which fields form the current schema.                                        |
| Services/AuthService.cs            | BCrypt password hashing, JWT creation and account verification/reset token logic.                         |
| Services/EventService.cs           | Venue-overlap and event-capacity validation.                                                              |
| Services/BookingService.cs         | Transactional booking creation, seat/parking validation, Pending hold, cancellation and resource release. |
| Workers/BookingExpirationWorker.cs | Automatic expiration of unpaid Pending bookings.                                                          |
| Services/PaymentService.cs         | Simulated payment, one-payment rule, booking confirmation and receipt data.                               |
| Frontend-UI/js/api.js              | Shared Fetch wrapper and JWT header handling.                                                             |
| Frontend-UI/js/seat-map.js         | Visual seat selection and booking initiation.                                                             |
| Frontend-UI/js/parking-map.js      | Optional parking selection.                                                                               |
| Frontend-UI/js/payment.js          | Booking hold countdown and payment call.                                                                  |

## 21. Development, Run and Database Setup

A practical setup sequence for the source snapshot is:

1. Install a .NET 9 SDK and Microsoft SQL Server / SQL Server Express on the development machine.
2. Open Backend-API/EventParking.sln in Visual Studio or enter the Backend-API directory in a terminal.
3. Update ConnectionStrings:DefaultConnection to a SQL Server instance accessible from that machine.  
The uploaded value is machine-specific and should not be reused blindly.
4. Restore NuGet packages (Visual Studio restore or dotnet restore).
5. Apply EF Core migrations to create/update the database. In Visual Studio Package Manager Console use Update-Database; with CLI tooling use dotnet ef database update.
6. Run the API using the HTTPS profile. The source config expects https://localhost:7283 and opens /swagger in Development.
7. Serve Frontend-UI using a local static web server or IDE live-server feature. js/api.js currently points to https://localhost:7283/api.

8. Register a customer, verify the account token flow as configured, log in, and use Swagger/browser flows to test events, seats, parking, booking and payment.

**Validation limitation:** This environment did not have the dotnet CLI installed, so the uploaded solution could not be compiled here. The stack/version analysis and endpoint inventory were performed by static inspection of the source files, migrations and configuration.

## 22. Git / GitHub and Source Control

The uploaded ZIP contains a Git repository with a GitHub origin, main and develop branches, plus feature branches for backend initialization, folder architecture, auth/customer work, events, seats, parking, bookings, payments and dashboards. This demonstrates a feature-branch workflow suitable for a multi-developer student project.

The .gitignore is based on the Visual Studio template and ignores build outputs, .vs metadata, NuGet packages, SQL Server data files and user-specific settings. However, the uploaded ZIP itself includes .git and .vs directories, which should normally not be included in a final source-code submission archive if the evaluator only needs the project files.

## 23. BRD Alignment and Implementation Notes

The project broadly implements the BRD stack and most functional modules. The following points are important before treating the project as a final, production-style submission.

| Area                         | Observation / Action Before Final Submission                                                                                                                                                         |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SQL database script          | No .sql file exists in the uploaded source. The BRD deliverable requires a full schema/seed script. EF migrations exist, but that is not the same deliverable.                                       |
| README                       | README.md contains only the project title and does not yet provide setup instructions, stack, ER diagram or module descriptions.                                                                     |
| Email delivery               | SMTP settings are placeholders; no email service sends verification or reset links. The forgot-password method only generates/stores a token.                                                        |
| Resend verification endpoint | The BRD specifies POST /api/auth/resend-verification; no such controller action exists in the current source.                                                                                        |
| Token storage                | Verification/reset tokens are stored as raw strings; the BRD calls for hashed token storage.                                                                                                         |
| Customer authorization       | The BRD marks customer search/deactivate/reactivate as admin-only, but CustomersController currently applies only [Authorize], so any authenticated account can reach them.                          |
| Ownership checks             | Profile GET/PUT and status endpoints accept arbitrary customer IDs and do not compare them to the JWT subject in the current controller/service implementation.                                      |
| Seat-map capacity rule       | SeatService.GenerateSeatMapAsync() compares generated seat count to Venue.Capacity, while the BRD states it must match the event's configured Capacity.                                              |
| Customer/event foreign keys  | The BRD schema expects Bookings to reference Customer and Event by FK. The current Booking stores CustomerEmail and derives event information from booked seats; Payments also stores CustomerEmail. |
| Notifications FK             | Notification has CustomerId, but its migration creates no foreign-key constraint to Customers.                                                                                                       |

| Area                          | Observation / Action Before Final Submission                                                                                                                                                    |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Parking uniqueness            | The database has a unique index on ParkingReservation.BookingId, not ParkingSlotId. Slot exclusivity is therefore enforced by status checks rather than a DB unique constraint.                 |
| Background expiry transaction | The worker does not explicitly wrap expiry/release in a transaction, although the BRD asks for transactional protection against simultaneous payment.                                           |
| CORS / secrets                | CORS is AllowAnyOrigin/Method/Header and the JWT secret is stored in appsettings.json. Fine for local demo only; not production safe.                                                           |
| Frontend notifications assets | notifications.html references css/style.css and js/customer/notifications.js, but those files do not exist in the uploaded tree; the available notification script is js/auth/notifications.js. |
| Admin parking delete          | admin-parking.js calls /parking/{slotId}; the backend delete endpoint requires /events/{eventId}/parking-slots/{slotId}.                                                                        |
| Profile localStorage naming   | profile.js expects customer_id and removes jwt_token, but login.js stores jwtToken and does not store customer_id. This makes the current profile flow inconsistent.                            |
| Ticket price model            | The BRD describes event ticket price, but Event currently has no ticket-price field. Seat price is supplied when generating the seat map and is used for totals.                                |
| Receipt download              | The API returns receipt DTO JSON. It does not generate a PDF receipt; customer-dashboard currently displays it via alert.                                                                       |

## 24. Deliverables Readiness Checklist

| BRD Deliverable               | Current Snapshot                                                                                                                          | Status                            |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
| Source Code                   | ASP.NET Core API + HTML/CSS/JS frontend are present in a Git repository.                                                                  | Substantially present             |
| SQL Database Script           | No .sql file found. EF Core migrations are present.                                                                                       | Missing required script           |
| REST API                      | Many CRUD/workflow endpoints are implemented with JWT/admin protection, but several BRD routes/authorization rules differ or are missing. | Partially complete                |
| Frontend Application          | Customer/admin pages, seat map and parking map are present; some path/endpoint inconsistencies remain.                                    | Substantially present; fix issues |
| Swagger Documentation         | Swagger/OpenAPI is configured with JWT bearer support in Development.                                                                     | Present                           |
| README File                   | README contains only a title.                                                                                                             | Incomplete                        |
| Full Technical Stack Document | This document covers actual .NET, frontend, backend, database, languages, APIs/services, packages, architecture and implementation notes. | Prepared                          |

## 25. Conclusion

The technical foundation of the project is appropriate for the BRD: a .NET 9 ASP.NET Core Web API, SQL Server persistence through Entity Framework Core, and a responsive vanilla HTML/CSS/JavaScript frontend using Fetch. JWT authentication, BCrypt hashing, Swagger, migrations, background booking expiry, transactions and role-based admin routes make the project more complete than a simple CRUD demo.

For submission readiness, the most important remaining tasks are to produce the required SQL schema/seed script, complete the README, finish real email delivery or clearly document it as simulated, correct the identified authorization/frontend route issues, align seat capacity with Event.Capacity, and harden secrets/CORS. After those changes, run a full build plus end-to-end tests for registration -> verification -> login -> event -> seats -> optional parking -> pending hold -> payment -> notification/cancellation/expiry.

## Source Basis

- Event & Parking Reservation System - Project BRD (v1.3), 18 pages, uploaded with this request.
- Event-Parking-System.zip source snapshot uploaded with this request; inspected directly, including .csproj, Program.cs, controllers, services, models, DTOs, migrations, appsettings, frontend pages/scripts and Git metadata.
- No outside web sources were used to determine the actual project stack or implementation details.